DocuGraph AI

Software Requirements Specification (SRS)

Field	Value
Version	1.1 (updated from Master Build Spec v1.0, TRD section)
Date	October 18, 2026
Status	Phase 1 complete and verified locally. Phase 2 next.
Companion	DocuGraph_AI_PRD_v1.1.pdf (product scope, metrics, roadmap)

1. Handoff Notes (Read First if Continuing in VS Code / Another AI)

- **Phase 1 is done:** FastAPI skeleton, pydantic-settings config, Docker Compose (FastAPI + Neo4j + Qdrant), /health. Verified: /health returns 200, Neo4j started with Bolt on 7687, browser on 7474.
- **Next task:** Phase 2, ingestion pipeline (services/ingest.py, services/extraction.py, POST /ingest).
- **Decision made:** fallback LLM is **Google Gemini (free tier)**, not Anthropic Claude. Groq stays primary. Keep all LLM calls behind one small client interface in services/extraction.py and services/synthesis.py so providers are swappable.
- **Decision made:** Qdrant is self-hosted via Docker, so QDRANT_API_KEY stays blank.
- **Secrets:** real values go only in .env (gitignored). .env.example holds placeholders. requirements.txt is only the pip package list. config.py declares fields with safe defaults; never hardcode secrets there.
- **Neo4j password** must be at least 8 characters or the container exits with code 70. If the password was changed after first start, run docker-compose down -v (wipes the Neo4j volume) and start again.
- **Docker:** Docker Desktop must be running before docker-compose up. The obsolete 'version:' line has been removed from docker-compose.yml.
- **Rules for the assistant:** follow the folder structure in section 11 exactly, build phase by phase, add tests with each phase, keep the MVP simple (single FastAPI app, no auth, no caching layers).

2. Introduction

DocuGraph AI is a hybrid document intelligence system. It ingests a PDF, builds a vector index (Qdrant) and a knowledge graph (Neo4j), answers natural-language questions by fusing both retrieval paths, and returns answers with citations and confidence scores. This SRS defines the functional and non-functional requirements, interfaces and data models for the MVP.

3. System Architecture

```
LAYER 1  INGESTION
  PDF parser (PyMuPDF) -> chunker (300-500 tokens, 50 overlap) -> chunks + page metadata

LAYER 2  DUAL EXTRACTION AND INDEXING (parallel)
  Extraction : NER + LLM relationship extraction -> triplets with confidence 0-1
  Vector     : sentence-transformers all-MiniLM-L6-v2 (384-dim) -> Qdrant
```

```
  Graph      : triplets -> Neo4j (nodes: type, name, confidence, page; edges: predicate, confidence, chunk_id)

LAYER 3  QUERY PROCESSING (parallel retrieval)
  Vector path : embed query -> Qdrant top-5
  Graph path  : NL -> Cypher -> Neo4j traversal (up to 3 hops)
  Fusion      : merge, rank by relevance + confidence

LAYER 4  SYNTHESIS
  LLM (Groq, fallback Gemini) -> answer with [chunk_id] markers -> citation parse -> page lookup

LAYER 5  FRONTEND
  React + TypeScript: answer display, vis.js graph, citation panel, metrics dashboard
```

4. Technology Stack

Component	Tool	Notes
Backend framework	FastAPI 0.104+, Python 3.11+	Async; pydantic 2 validation
PDF parsing	PyMuPDF (fitz)	In-memory parsing; no PDF stored
Entity recognition	spaCy 3.7+ (optional)	Person, Org, Date, Amount. Can be dropped if the LLM handles entities and relationships in one pass
Primary LLM	Groq API	Extraction + synthesis; free tier
Fallback LLM	**Google Gemini API (google-genai SDK)**	Free tier; replaces Anthropic Claude. Confirm current model name at implementation
Embeddings	sentence-transformers all-MiniLM-L6-v2	384-dim, local
Vector DB	Qdrant (self-hosted Docker)	Cosine, top-k=5, no API key
Graph DB	Neo4j 5.10 Community (Docker)	Cypher, up to 3 hops, uniqueness constraints, name indexes
Frontend	React 18, TypeScript 5, Vite, Tailwind 3, vis.js, Axios	Phase 6
Deploy	Docker Compose (local), Railway (backend), Vercel (frontend)	Phase 7

5. Functional Requirements

ID	Requirement	Phase
FR-1	The system shall expose GET /health returning service status.	1 (done)
FR-2	The system shall accept a PDF upload (max 50 MB) with a document_id and parse it into page-tagged text.	2
FR-3	The system shall split text into 300-500 token chunks with 50-token overlap, preserving page number and offsets.	2
FR-4	The system shall extract entities and relationships from each chunk as triplets with a confidence score 0-1.	2
FR-5	The system shall embed chunks and index them in Qdrant with payload chunk_id, page_number, text, document_id.	3
FR-6	The system shall insert entities and relationships into Neo4j with source page, chunk ids and confidence.	3
FR-7	The system shall compute and expose extraction quality statistics per document.	3
FR-8	The system shall run vector and graph retrieval in parallel and fuse results by relevance and confidence.	4

ID	Requirement	Phase
FR-9	The system shall translate natural-language questions to Cypher with validation (read-only queries only).	4
FR-10	The system shall synthesize an answer using only retrieved context, with [chunk_id] citations.	5
FR-11	The system shall map every citation to chunk id, page and confidence, and report citation coverage.	5
FR-12	The UI shall render answer, citations, an interactive graph (200+ nodes) and a metrics dashboard.	6
FR-13	The system shall degrade gracefully per the fallback table in section 9.	2-5

6. API Specification

POST /ingest (Phase 2)

```
Request (multipart/form-data): document_id (alphanumeric), file (PDF, max 50 MB), document_type (optional)
Response 200:
{ "status": "success", "document_id": "contract_001", "chunks_created": 127,
  "entities_extracted": 45, "relationships_extracted": 82, "avg_confidence": 0.89,
  "pages": 15, "processing_time_seconds": 8.2 }
Error: { "status": "error", "error": "PDF parsing failed", "details": "..." }
```

POST /query (Phase 4-5)

```
Request: { "document_id": "contract_001", "query": "...(max 1000 chars)...",
           "retrieval_method": "hybrid" | "vector" | "graph" }
Response 200:
{ "status": "success", "answer": "...",
  "citations": [ { "text": "...", "chunk_id": 42, "page": 3, "confidence": 0.96, "source": "vector" } ],
  "graph_subgraph": { "nodes": [ {"id": "alice", "type": "Person", "confidence": 0.95} ],
                      "edges": [ {"source": "alice", "target": "contract_001",
                                  "predicate": "signs", "confidence": 0.96} ] },
  "retrieval_stats": { "vector_results": 5, "graph_results": 3, "fusion_score": 0.95, "latency_seconds": 1.2 } }
```

GET /documents/{document_id}/stats (Phase 3)

```
Response: { "document_id": "...", "status": "indexed",
  "extraction_quality": { "entities_by_type": {...}, "avg_confidence_by_type": {...},
                          "relationships_by_predicate": {...}, "overall_quality_score": 0.91 } }
```

GET /health

Phase 1 (current): returns {"status": "healthy", "version": "0.1.0"}. Phase 3 extends it to report groq/gemini, neo4j and qdrant connectivity:

```
{ "status": "healthy", "services": { "llm": "connected", "neo4j": "connected", "qdrant": "connected" } }
```

7. Data Models

Chunk (Qdrant point)

```
{ "id": "chunk_42", "document_id": "contract_001", "page_number": 3, "text": "...",
  "embedding": [384 floats], "start_offset": 1200, "end_offset": 1800,
  "chunk_metadata": { "section": "Parties", "confidence": 0.95 } }
```

Neo4j

```
(:Person {name, confidence, source_page, source_chunk_id, occurrences, aliases[]})
(:Amount {value, currency, confidence, context})
(a)-[:SIGNS    {confidence, source_chunk_ids[], source_pages[], predicate_confidence}]->(b)
```

```
(a)-[:PAYS     {amount, currency, frequency, confidence}]->(b)
Constraints: unique (type, name). Indexes: entity name.
```

Document record

```
{ "document_id", "title", "content_hash", "num_pages", "ingestion_timestamp", "document_type", "status" }
```

8. Non-Functional Requirements

8.1 Performance targets

Operation	Target
PDF parsing	under 1 s per 100 pages
Extraction	under 0.5 s per chunk (with LLM latency; batch and cache)
Embedding	under 0.1 s per chunk (local model)
Neo4j insertion	under 0.05 s per triplet (batch insert)
Vector search / graph search	under 100 ms / under 200 ms
Total ingestion	8-10 s per 100 pages (MVP acceptable)
Total query	under 2 s (goal 1.5 s)

Note: the original spec also said 'handle 500-page PDFs in under 2 seconds' as an NFR. That conflicts with the ingestion target above. Interpret the 2 s figure as **query latency** on an already-indexed 500-page document.

8.2 Security

- All secrets in .env locally and in Railway/Vercel secrets in deployment; .env is gitignored; never log keys or full extracted data.
- Neo4j default credentials changed (done via NEO4J_PASSWORD, min 8 chars); databases reachable only from the FastAPI container (do not expose ports publicly in deployment).
- Input validation: PDF max 50 MB, query max 1000 chars, document_id alphanumeric only (blocks path traversal).
- CORS restricted to the Vercel frontend origin before deployment (currently '*' for local dev).
- Rate limit 100 requests/minute per IP.
- LLM-generated Cypher must be validated as read-only (no CREATE/MERGE/DELETE/SET/DROP) before execution.
- Structured JSON logging; log query count and latency only.
- Stateless processing: PDFs are not stored; only extracted data persists in the databases. (Audit trail, encryption at rest and 30-day retention are future work.)

9. Error Handling and Fallbacks (Updated)

Component	Failure	Fallback
Groq API	Rate limited / error	Retry with backoff, then **Gemini free tier**
Gemini API	Throttled / error	Use spaCy-only extraction (lower quality) for ingestion; return clear error for synthesis
Neo4j	Connection fails	Vector-only retrieval (less reasoning)
Qdrant	Connection fails	Graph-only retrieval (less semantic)

Component	Failure	Fallback
LLM generation	Low-confidence or unsupported claims	Filter by confidence threshold; answer only from retrieved context

10. Configuration

.env variables (real values in .env only)

Variable	Purpose	Value guidance
FASTAPI_ENV, DEBUG	App mode	development, true
GROQ_API_KEY	Primary LLM	From console.groq.com
GEMINI_API_KEY	Fallback LLM (**new; replaces ANTHROPIC_API_KEY**)	From Google AI Studio. Add to config.py and .env.example when Phase 2 starts
NEO4J_URI / NEO4J_USER	Graph DB connection	bolt://neo4j:7687 / neo4j
NEO4J_PASSWORD	Graph DB password	Min 8 characters
QDRANT_URL	Vector DB	http://qdrant:6333
QDRANT_API_KEY	Cloud only	Leave blank for self-hosted
REACT_APP_API_URL	Frontend to backend (Phase 6; with Vite use VITE_API_URL)	http://localhost:8000

Local run

```
cp .env.example .env          # then edit .env with real values
docker-compose up --build     # Docker Desktop must be running
curl http://localhost:8000/health
Neo4j browser : http://localhost:7474   (bolt://localhost:7687, user neo4j)
Qdrant UI     : http://localhost:6333/dashboard
```

Dependency notes for Phase 2+

- Add to requirements.txt as each phase needs it: PyMuPDF, python-multipart, sentence-transformers, qdrant-client, neo4j, groq, google-genai, python-json-logger (and spaCy if kept).
- The original spec pinned qdrant-client==2.7.0, which does not exist (the client is on 1.x). Install the current 1.x release and pin what actually installs. Re-verify all old pins at install time.
- Remove the anthropic dependency; the fallback is Gemini.

11. Folder Structure

```
docugraph-ai/
  README.md  .gitignore  .env.example  docker-compose.yml
  backend/
    main.py  config.py  requirements.txt  Dockerfile
    api/       __init__.py  routes.py  models.py
    services/  __init__.py  ingest.py  extraction.py  embedding.py
               neo4j_handler.py  qdrant_handler.py  query_router.py  synthesis.py
    utils/     __init__.py  logger.py  validators.py  constants.py
    tests/     __init__.py  test_ingest.py  test_extraction.py  test_query.py  test_integration.py
  frontend/    (Vite + React + TS: src/pages, src/components, src/services/api.ts, src/types)
  docs/        ARCHITECTURE.md  API.md  SETUP.md  DEPLOYMENT.md  PRD.md  TRD.md  BENCHMARKS.md
  benchmarks/  data/sample_docs  data/questions.json  eval.py  metrics.py  results.json
  PROJECT_DIARY.md
```

Phase 1 currently contains: main.py, config.py, requirements.txt, Dockerfile, api/ (empty router and models), empty services/, utils/, tests/ packages, docker-compose.yml, .env.example, .gitignore, README.md.

12. Implementation Plan and Status

Phase	Scope	Days	Status
1	FastAPI skeleton, env config, Docker Compose, /health	1-3	DONE
2	PDF parsing (PyMuPDF), chunking, entity extraction, relationship extraction (Groq, Gemini fallback), confidence scoring, POST /ingest	4-6	Next
3	Neo4j triplet insertion, Qdrant indexing, metadata, query optimization, /documents/{id}/stats, full /health	7-9	Planned
4	Vector search, NL-to-Cypher, graph search, fusion, POST /query (retrieval part)	10-12	Planned
5	LLM synthesis, citation extraction, page lookup, confidence annotation	13-15	Planned
6	React + Vite, query form and results, vis.js graph, citation panel, metrics page	16-18	Planned
7	End-to-end tests, bug fixes, performance, benchmarks, deploy to Railway and Vercel	19-21	Planned

13. Testing Strategy

- **Unit:** PDF to chunks, extraction to triplets, embedding and indexing, Neo4j operations, fusion ranking, synthesis format.
- **Integration:** PDF upload to ingestion to query to answer; 5 document types; empty or malformed PDFs; rate limiting; concurrent requests.
- **Benchmark:** 50 hand-written Q&A; pairs over 10 documents; score accuracy (human), citation coverage and latency. Expected: accuracy 85-95%, citation coverage 95%+, latency 1-3 s.
- Test every component as it is built; update PROJECT_DIARY.md weekly.

14. Acceptance Criteria (Definition of Done)

- Backend runs locally without errors (Phase 1: met).
- Neo4j and Qdrant index documents correctly.
- Hybrid query returns both vector and graph results; citations link to source chunks.
- Graph visualization renders 200+ nodes; React UI loads and submits queries.
- Backend deployed on Railway and frontend on Vercel; live demo URL works.
- Benchmark script runs; 50 Q&A; pairs evaluated; accuracy at least 85% (stretch 94%).
- API, setup and architecture docs complete; README with references.

15. Change Log vs. Master Build Spec v1.0

- Fallback LLM: Anthropic Claude replaced by Google Gemini free tier (GEMINI_API_KEY; google-genai).
- Phase 1 delivered; docker-compose parameterized with NEO4J_PASSWORD; obsolete compose 'version' key removed.
- Qdrant API key clarified as not applicable for self-hosted.

- Corrected invalid qdrant-client pin; flagged the 500-page performance NFR ambiguity; accuracy figures reframed as targets.
- Added requirement that LLM-generated Cypher be validated read-only.